Assignment 2: Codebreaker Optimization

Due: 10/02/2026, 11:59 PM
Submission: On Canvas

Overview

In this assignment, you will formulate and solve a constraint satisfaction problem using local search and optimization. You will implement and compare Hill Climbing and Simulated Annealing to recover a hidden numeric code from a collection of clues. The goal is to understand how a constraint satisfaction problem can be converted into an optimization problem, how the choice of neighborhood affects local search, and how Simulated Annealing can escape local optima that may prevent Hill Climbing from finding a solution.

Problem: Codebreaker

A system has selected a secret numeric code consisting of N digits, where each digit is an integer from 0 through 9. You do not know the secret code. Instead, you are given several previous guesses together with feedback describing how closely each guess matches the secret code. For every guess, the system reports:

- **Exact matches:** digits that have both the correct value and the correct position.
- **Misplaced matches:** digits that occur in the secret code but appear in a different position.

For example, suppose the secret code is $[4, 1, 7, 3]$ and the guess is $[4, 7, 5, 1]$. The feedback is Exact = 1, Misplaced = 2. The digit 4 is in the correct position, while digits 7 and 1 occur in the secret code but appear in different positions. Your goal is to find **any code that satisfies every provided clue**.

Part 1: State Representation

Represent each candidate state as $s = [d_1, d_2, \ldots, d_N]$, where each $d_i \in \{0, 1, \ldots, 9\}$. For example, a candidate six-digit code might be $[3, 8, 1, 4, 7, 2]$. Implement the function `random_state(code_length)` that generates and returns a random candidate code of the requested length. Digits may appear more than once.

Part 2: Evaluating a Guess

Implement the function `evaluate_guess(candidate, guess)`. The function should compare a candidate code against a guess and `return exact_matches, misplaced_matches`. A digit may only be matched once. For example, consider `candidate` $= [1, 1, 2, 3]$ and `guess` $= [1, 2, 1, 4]$. The result should be Exact = 1, Misplaced = 2. Your implementation must correctly handle repeated digits.

Part 3: Objective Function

Each clue consists of (`guess`, `expected_exact`, `expected_misplaced`). For example, $([1, 2, 3, 4], 1, 1)$ means that the secret code must produce exactly one exact match and one misplaced match when compared with $[1, 2, 3, 4]$. For clue i, let E_i and M_i denote the expected numbers of exact and misplaced matches. For a candidate state s, let $\widehat{E}_i(s)$ and $\widehat{M}_i(s)$ denote the feedback generated by that candidate. Define the cost of a candidate as

$$Cost(s) = \sum_i \left(|E_i - \widehat{E}_i(s)| + |M_i - \widehat{M}_i(s)| \right).$$

Implement `cost(candidate, clues)`. A candidate with $Cost(s) = 0$ satisfies every clue and is therefore a valid solution. The optimization problem is therefore $\min_s Cost(s)$.

Part 4: Neighborhood Function

Local search algorithms move between neighboring candidate solutions. For this assignment, two states are neighbors if they differ in **exactly one digit**. For example, Current = $[4, 1, 7, 3, 8, 2]$ and Neighbor = $[4, 1, 5, 3, 8, 2]$. Implement `get_neighbors(state)` that returns all possible one-digit changes to the current state. For a code of length N, there should be $9N$ neighbors because each position can be changed to any of the other nine digits. Also implement `random_neighbor(state)` which randomly selects one position and changes it to a different digit. You will use the complete neighborhood for Hill Climbing and a randomly selected neighbor for Simulated Annealing.

Part 5: Hill Climbing

Implement `hill_climb(clues, code_length, max_steps)` using steepest-descent Hill Climbing. The algorithm should:

1. generate a random initial state,
2. compute its cost,
3. generate all neighboring states,
4. select the neighbor with the lowest cost,
5. move to that neighbor only if it improves the current cost, and
6. repeat until a stopping condition is reached.

Stop when a solution with $Cost = 0$ is found, no neighboring state improves the current state, or `max_steps` has been reached. Your function should `return solution, final_cost, iterations`. For example, one run might produce:

Initial State:	$[8, 2, 1, 9, 3, 7]$
Initial Cost:	16
Final State:	$[4, 1, 7, 9, 8, 2]$
Final Cost:	2
Iterations:	11

A final cost greater than zero means that Hill Climbing did not find a valid solution.

Part 6: Simulated Annealing

Hill Climbing only accepts improvements, which can cause the algorithm to become stuck at a local optimum. Simulated Annealing sometimes accepts worse solutions, allowing the search to escape these regions. Implement `simulated_annealing(clues, code_length, max_steps, initial_temperature, cooling_rate)`. At each iteration, generate one random neighbor and compute $\Delta = Cost(new) - Cost(current)$. If $\Delta < 0$, the new state improves the solution and should always be accepted. If $\Delta \geq 0$, accept the new state with probability $P = e^{-\Delta/T}$, where T is the current temperature. After each iteration, update the temperature according to $T \leftarrow \alpha T$, where α is the cooling rate. Your function should `return solution, final_cost, iterations`. Stop when $Cost = 0$, `max_steps` is reached, or the temperature becomes sufficiently small.

Part 7: Generating Codebreaker Problems

Rather than using fixed secret codes, you will generate your own random secret codes and then attempt to recover them using Hill Climbing and Simulated Annealing. For each experiment:

1. randomly generate a secret code,
2. randomly generate a collection of guesses,
3. use the secret code to compute the correct feedback for each guess,
4. store the resulting guesses and feedback as the clue set, and
5. use only the clue set as input to the optimization algorithms.

The optimization algorithms should **not** have access to the secret code.

Part 8: Optimization Experiments

Test both Hill Climbing and Simulated Annealing on three problem sizes:

1. **Problem A: 4-Digit Code**. Randomly generate a secret code of length $N = 4$
2. **Problem B: 6-Digit Code**. Randomly generate a secret code of length $N = 6$
3. **Problem C: 8-Digit Code**. Randomly generate a secret code of length $N = 8$

For each algorithm, report:

- the original secret code,
- the code found by the algorithm,
- final cost,
- number of iterations, and
- runtime (using `time.perf_counter()`).

A zero-cost solution does not necessarily have to be identical to the original secret code. If multiple codes satisfy all generated clues, any zero-cost solution is valid. Run each problem case 10 times, i.e., generate 10 random codes and log your model's performance in the table below.

Code Length	Algorithm	Successful Runs	Success Rate	Avg. Final Cost	Avg. Iterations	Avg. Runtime
4	Hill Climbing					
4	Simulated Annealing					
6	Hill Climbing					
6	Simulated Annealing					
8	Hill Climbing					
8	Simulated Annealing					

Use the same generated problem for both algorithms before generating the next problem. This ensures that Hill Climbing and Simulated Annealing are evaluated on the same set of constraints.

Part 9: Visualizing the Optimization Process

Select one 8-digit problem from Part 9. For one run of each algorithm, record the current cost after every iteration. Create a plot with Iteration on the horizontal axis and Cost on the vertical axis. Your notebook should contain one Hill Climbing cost plot and one Simulated Annealing cost plot. Use these plots to help explain how the behavior of the two algorithms differs.

Part 10: Analysis Questions

Answer each question in approximately 2–4 sentences.

1. Why can the Codebreaker problem be represented as an optimization problem using the cost function defined earlier?
2. Why does a cost of zero indicate that a candidate satisfies the generated constraints?
3. Is a zero-cost solution guaranteed to be identical to the original secret code? Explain why or why not.
4. Hill Climbing may stop with a cost greater than zero even though a valid solution exists. Why can this happen?
5. Why does Simulated Annealing sometimes accept a neighboring state with a higher cost?
6. Compare the results for the 4-digit, 6-digit, and 8-digit problems. How did increasing the size of the state space affect the algorithms?
7. Compare the success rates of Hill Climbing and Simulated Annealing. What characteristics of the algorithms help explain the difference you observed?
8. What would happen if the temperature in Simulated Annealing decreased extremely quickly?
9. What would happen if the temperature remained high for most of the search?
10. The neighborhood in this assignment changes exactly one digit. How might the behavior of the algorithms change if a neighboring state were allowed to modify two digits at once?